Kaip dinamiškai keisti objekto išvaizdą nekeičiant funkcionalumo,

Kaip veiksmą paversti objektu, kad atidėti vykdymą ar atšaukti

T120B516 Objektinis programų projektavimas

2026 rudens semestras

Dr. Andrej Ušaniov

Jus esate čia

Kodėl tai svarbu? Nes jau greitai jūsų laukia kontrolinis

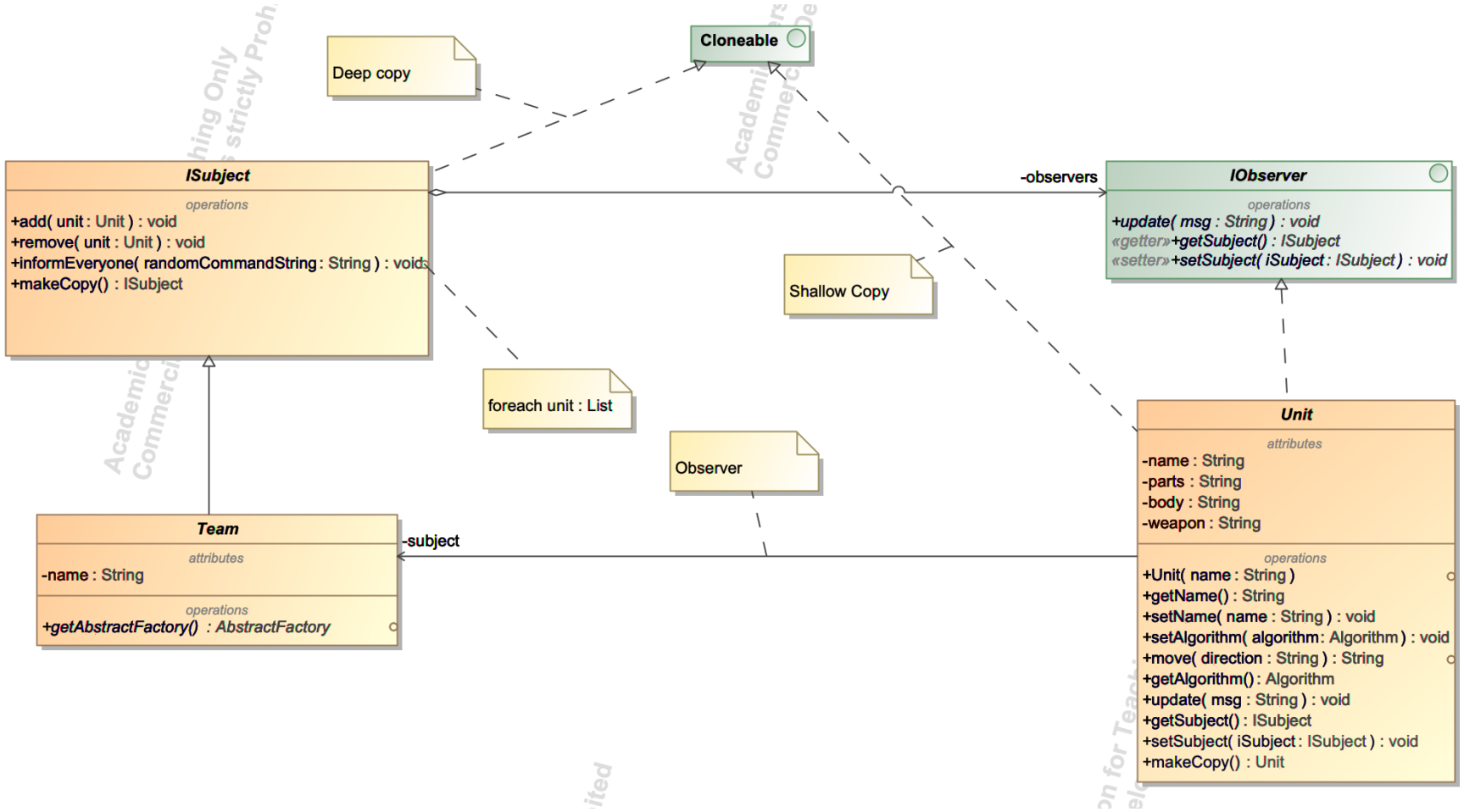
T120B516 Paskaitų planas 2026 m. rudens semestras

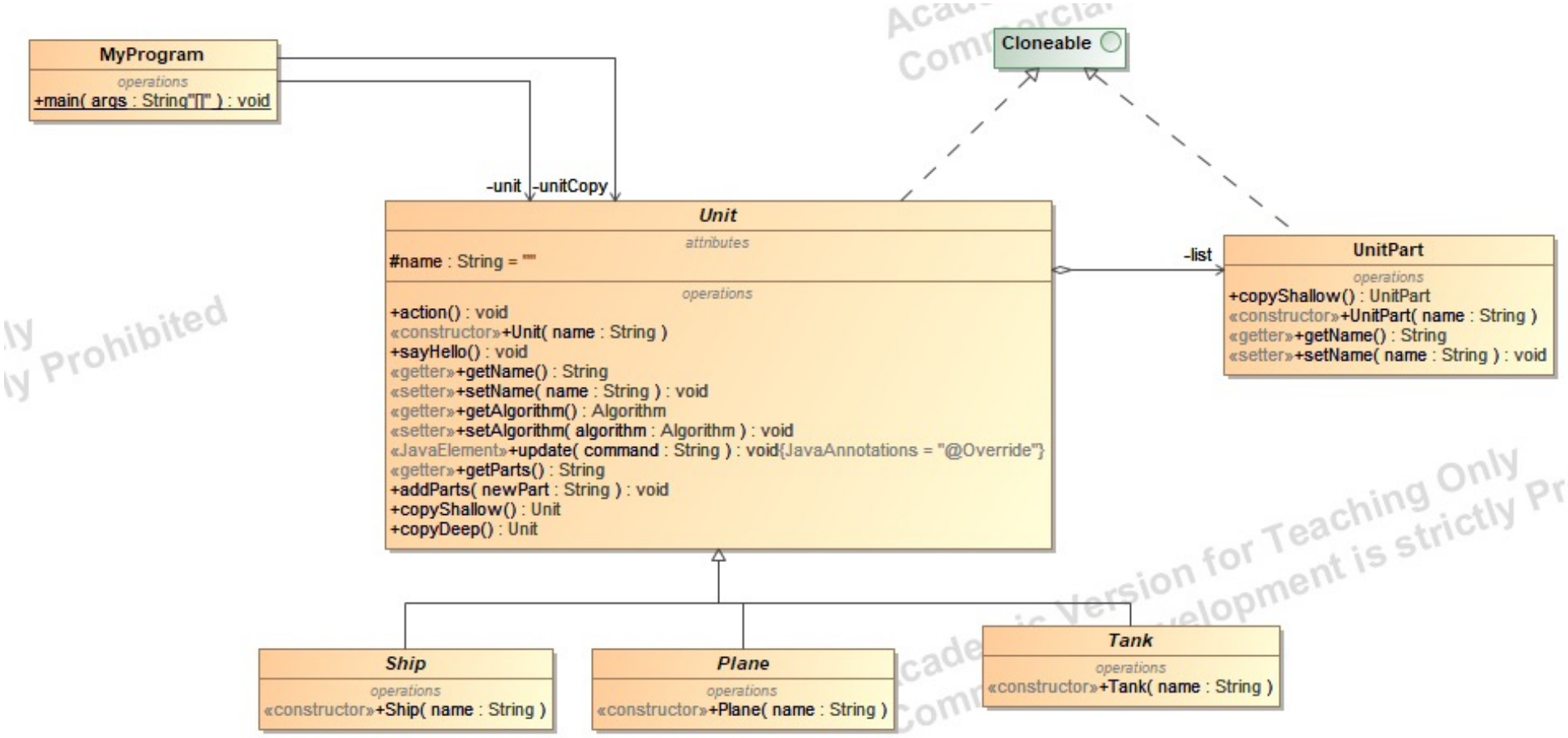
Sav. Nr.	Paskaitos data	Paskaitų temos	Projektinio darbo veiklos	Projektinės veiklos ir testo periodas	Testai
1			Sudaryti grupę, Suderint temą	iki 2026-09-08	
2	2026-09-07	Apie modulį, Įžanga į projektavimo šablonus, Hello World	Pristatyti projekto aprašymą, UML Use case, Class diagram, Activity diagram, Reikalavimai	iki 2026-09-15	
3	2026-09-14	*Projektavimo šablonų apžvalga, Signal-R apžvalga, UML ir OOP principai*	Pristatyti žaidimo prototipą (tinklo komunikacija, grafika, valdymas), Klasių diagrama	iki 2026-09-22	
4	2026-09-21	01-Singleton, 02-Factory, 03-Abstract Factory	Šablonų taikymas: 01-Singleton, 02-Factory, 03-Abstract Factory, 04-Strategy, 05-Observer, 06-Builder, 07-Prototype	iki 2026-09-29	
5	2026-09-28	04-Strategy, 05-Observer		iki 2026-10-06	
6	2026-10-05	06-Builder, 07-Prototype		iki 2026-10-13	
7	2026-10-12	08-Decorator, 09-Command	Šablonų taikymas: 08-Decorator, 09-Command, 10-Adapter, 11-Façade, 12-Bridge	iki 2026-10-20	
8	2026-10-19	10-Adapter, 11-Façade, 12-Bridge, Pasiruošimas K1		iki 2026-10-27	
9	2026-10-26	K1 per Moodle ir MS Teams su įjungta kamera	**P1 gynimas**	iki 2026-11-03	
10	2026-11-02	Vėlinės - paskaitos nebus	**P1 pakartotinas gynimas**	iki 2026-11-10	**T1**
11	2026-11-09	13-Template Method, 14-Iterator,	Šablonų taikymas: 13-Template method, 14-Iterator, 15-Composite, 16-Flyweight, 17-State, 18-Proxy, 19-ChainOfRespnsibility	iki 2026-11-17	
12	2026-11-16	15-Composite, 16-Flyweight		iki 2026-11-24	
13	2026-11-23	17-State, 18-Proxy, 19-ChainOfResponsibility		iki 2026-12-01	
14	2026-11-30	20-Interpreter, 21-Mediator	Šablonų taikymas: 20-Interpreter, 21-Mediator, 22-Memento, 23-Visitor	iki 2026-12-08	
15	2026-12-07	22-Memento, 23-Visitor		iki 2026-12-15	**T2**
16	2026-12-14	Pasiruošimas egzaminui, Projektinių darbų pristatymas	**P2 gynimas**	iki 2026-12-22	
17	2026-12-21	K1 perlaikymas auditorijoje	**P2 pakartotinas gynimas**	iki 2026-12-22 imtinai	
18-21	2027-01-??	Egzaminas			

Ką prisimenate iš praėjusio karto?

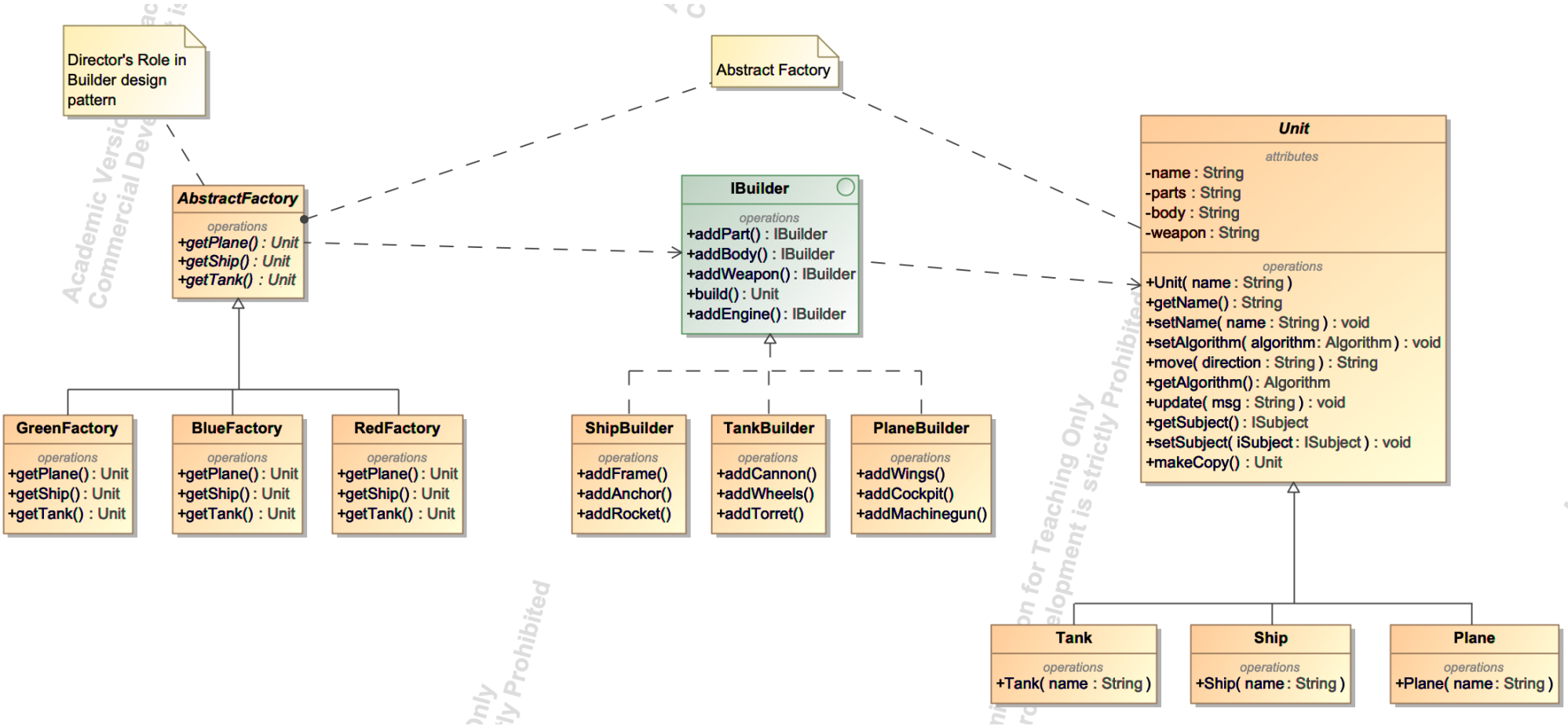
- **Builder** - Kuriame sudėtingą objektą žingsnis po žingsnio

- **Prototype** - Naują objektą sukuriame kopijuodami jau turimą

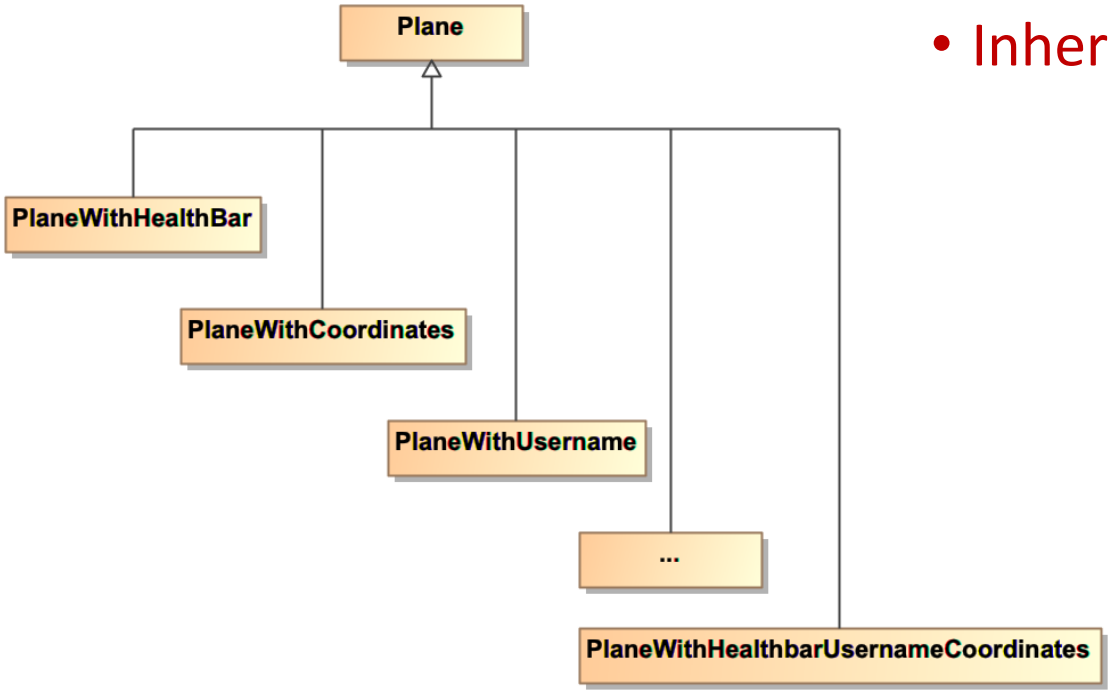

Deep copy
Cloneable
ISubject
operations
+add(unit : Unit) : void
+remove(unit : Unit) : void
+informEveryone(randomCommandString : String) : void
+makeCopy() : ISubject
-observers
IObserver
operations
+update(msg : String) : void
«getter»+getSubject() : ISubject
«setter»+setSubject(iSubject : ISubject) : void
Shallow Copy
foreach unit : List
Observer
Unit
attributes
-name : String
-parts : String
-body : String
-weapon : String
operations
+Unit(name : String)
+getName() : String
+setName(name : String) : void
+setAlgorithm(algorithm : Algorithm) : void
+move(direction : String) : String
+getAlgorithm() : Algorithm
+update(msg : String) : void
+getSubject() : ISubject
+setSubject(iSubject : ISubject) : void
+makeCopy() : Unit
Team
attributes
-name : String
operations
+getAbstractFactory() : AbstractFactory
-subject



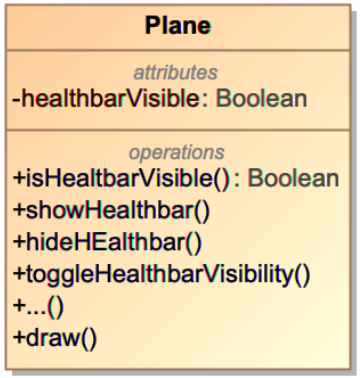
Prohibited
Version for Teaching Only


Director's Role in Builder design pattern
Abstract Factory
AbstractFactory
operations
+getPlane() : Unit
+getShip() : Unit
+getTank() : Unit
IBuilder
operations
+addPart() : IBuilder
+addBody() : IBuilder
+addWeapon() : IBuilder
+build() : Unit
+addEngine() : IBuilder
Unit
attributes
-name : String
-parts : String
-body : String
-weapon : String
operations
+Unit(name : String)
+getName() : String
+setName(name : String) : void
+setAlgorithm(algorithm : Algorithm) : void
+move(direction : String) : String
+getAlgorithm() : Algorithm
+update(msg : String) : void
+getSubject() : ISubject
+setSubject(iSubject : ISubject) : void
+makeCopy() : Unit
GreenFactory
operations
+getPlane() : Unit
+getShip() : Unit
+getTank() : Unit
BlueFactory
operations
+getPlane() : Unit
+getShip() : Unit
+getTank() : Unit
RedFactory
operations
+getPlane() : Unit
+getShip() : Unit
+getTank() : Unit
ShipBuilder
operations
+addFrame()
+addAnchor()
+addRocket()
TankBuilder
operations
+addCannon()
+addWheels()
+addTorret()
PlaneBuilder
operations
+addWings()
+addCockpit()
+addMachinegun()
Tank
operations
+Tank(name : String)
Ship
operations
+Ship(name : String)
Plane
operations
+Plane(name : String)

Turime žaidime lektuvą, norime rodyti koordinates, vardą, gyvybių lentelę, kūro kiekį... viską iškarto arba tik dalį



- Inheritance explosion problem



Decorator - Papildome objektą naujomis atsakomybėmis jo nekeisdami.

- Type: Structural
- Also Known As: Wrapper, Augmentor, Extender
- Intent: “Attach **additional responsibilities** to an object **dynamically**. Decorators provide a flexible alternative to subclassing for extending functionality”
- this design pattern is all about using *wrapper* code to extend your core code.

Decorator’s real life example

Vertėjas

Client
↓ LT
Translator
↓ EN
Service
↓ EN
Translator
↓ LT
Client



Kavos užsakymas



Turime bazinę juodą **kavą**. Klientas gali paprašyti įpilti **pieno**, įberti **cukraus** arba įpilti **sirupo**. Kiekvienas priedas padidina kavos **kainą** ir papildo aprašymą.

Ar objektas turi žinoti, kad jo duomenis kažkas pakeitė pakeliui?

Turime `Unit.move()` Ką dar norėtume padaryti aplink šį metodą?

log start

↓

check permission

↓

move()

↓

measure time

↓

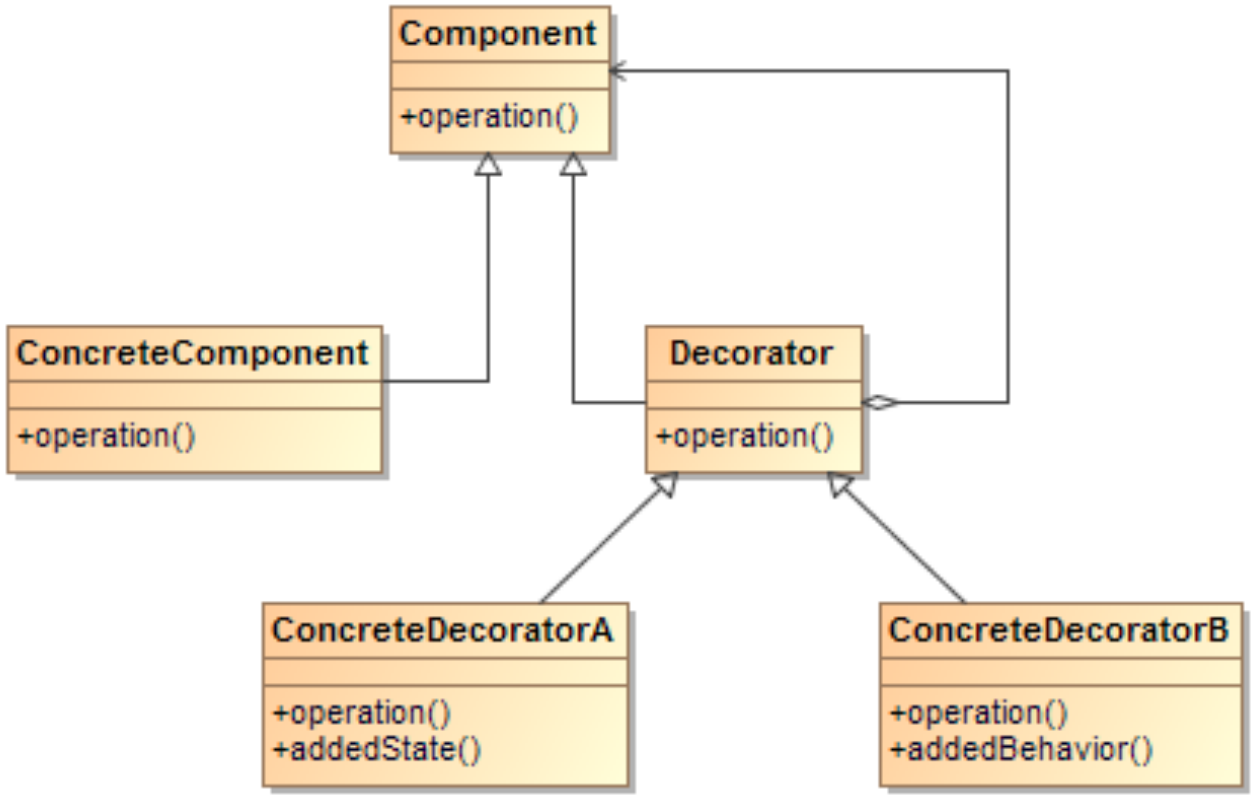
log end

Ar tikrai visa tai turi būti Unit.move() viduje?

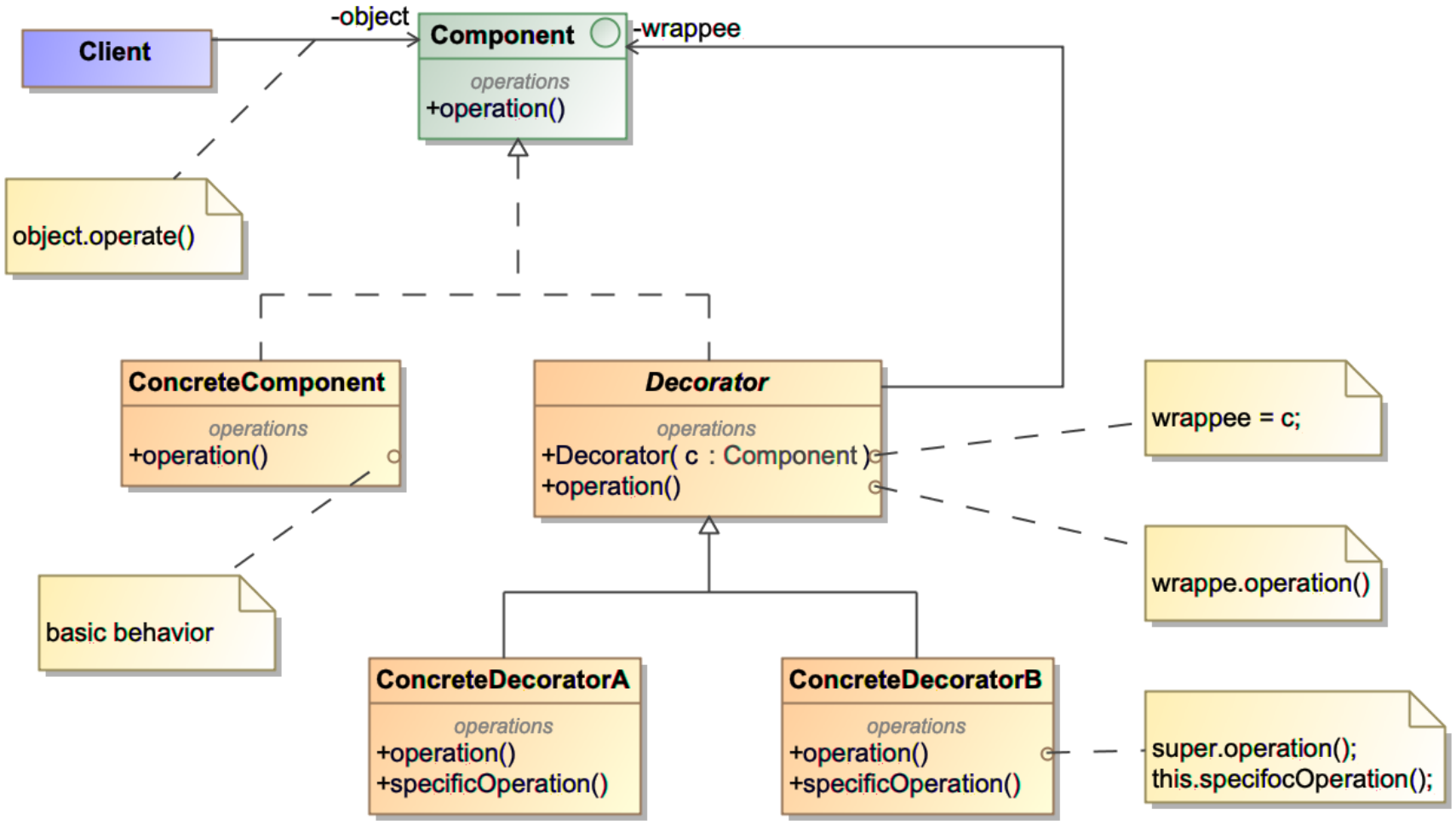
Kaip Unit.move() papildymo funkcionalumu problemą sprendžia GoF

Decorator's structure

- Decorator classes have to act as wrappers for the Component class
- By extending Decorator from Component we can ensure Decorator should be able to hold objects that wrap Component objects.



Pabandykime iš diagramos suprasti kaip veikia šablonas



Tai kokią problemą mes ką tik išsprendėme?

- norime papildyti **konkretų objektą**, o ne visą klasę;
- esamos klasės nenorime arba negalime keisti.
- funkcionalumą norime dinamiškai **runtime pridėti arba nuimti**;
- paveldėjimas sukurtų per daug kombinacijų;

Kiek sluoksnių galime uždėti vieną ant kito?

Ką klientas mato iš išorės?



Ar dekoratorių eiliškumas svarbus?

Variantas A:

```
Logging(
    Security(
        Unit
))
```

Variantas B:

```
Security(
    Logging(
        Unit
))
```

Ar loginsime bandymą atlikti veiksmą, jeigu naudotojas neturi teisių?

Kas nusprendžia, kokius Decorator uždėti?

A) pats Unit

B) Client

C) Factory

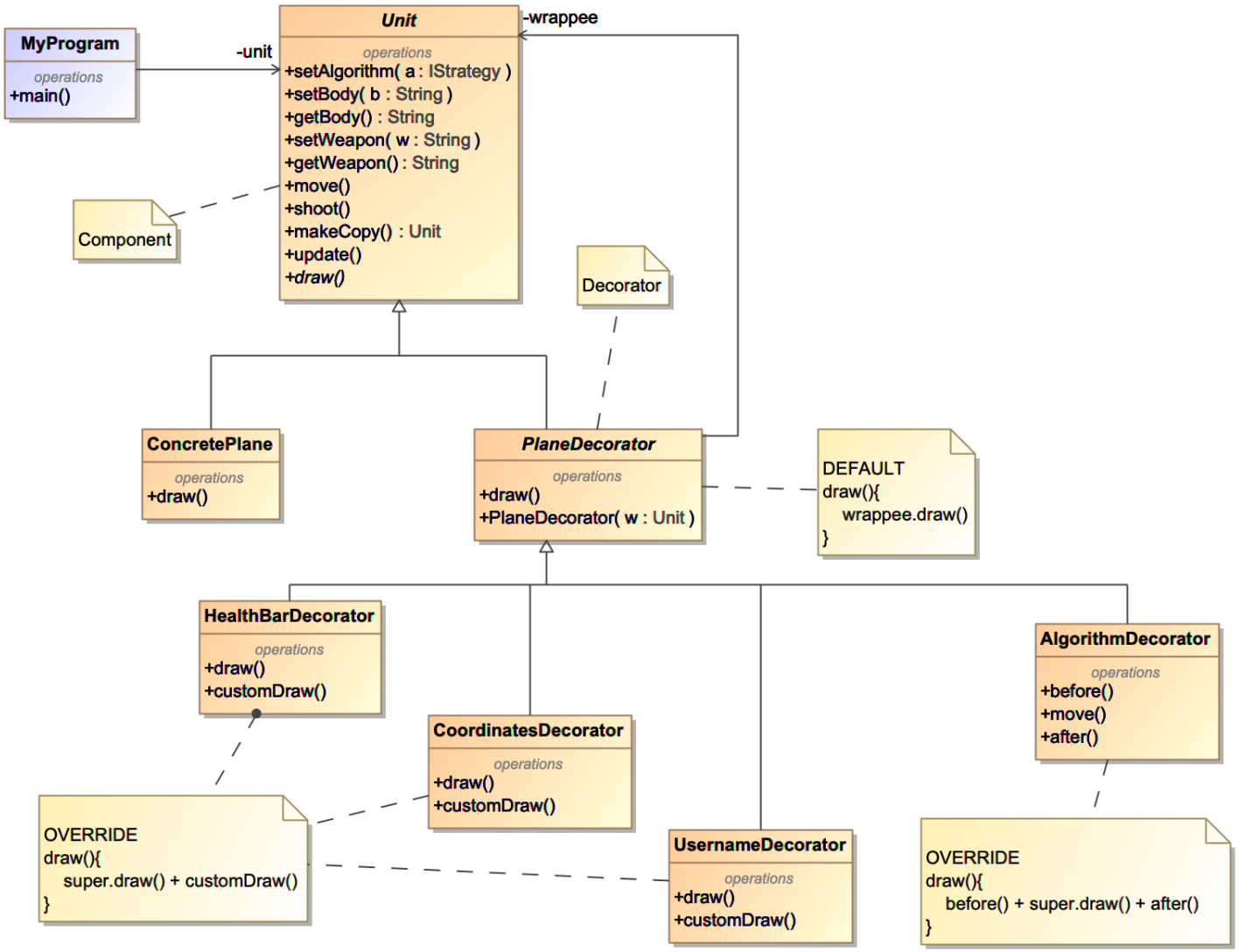
Ar tai vis dar Decorator?

```
public void move() {
    System.out.println("Access denied");
    // component.move()
}
```

O jei Component turi 20 metodų ar visus dekoruojame?

AbstractDecorator deleguoja visus metodus.

ConcreteDecorator perrašo tik reikalingus



Kur galima taikyti Dekorator jūsų žaidimuose?

- Security
- Timing
- Logging

Kas dar?

Kodėl visur nenaudojame Decorator?

✅ funkcionalumą galime kombinuoti

✅ nereikia naujų subclass kiekvienai kombinacijai

✅ galime runtime pridėti ar pašalinti sluoksnius

⚠️ daug mažų wrapper objektų

⚠️ sunku suprasti, kuris sluoksnis ką padarė

⚠️ sunku iš grandinės išimti konkretų vidurinį Decorator

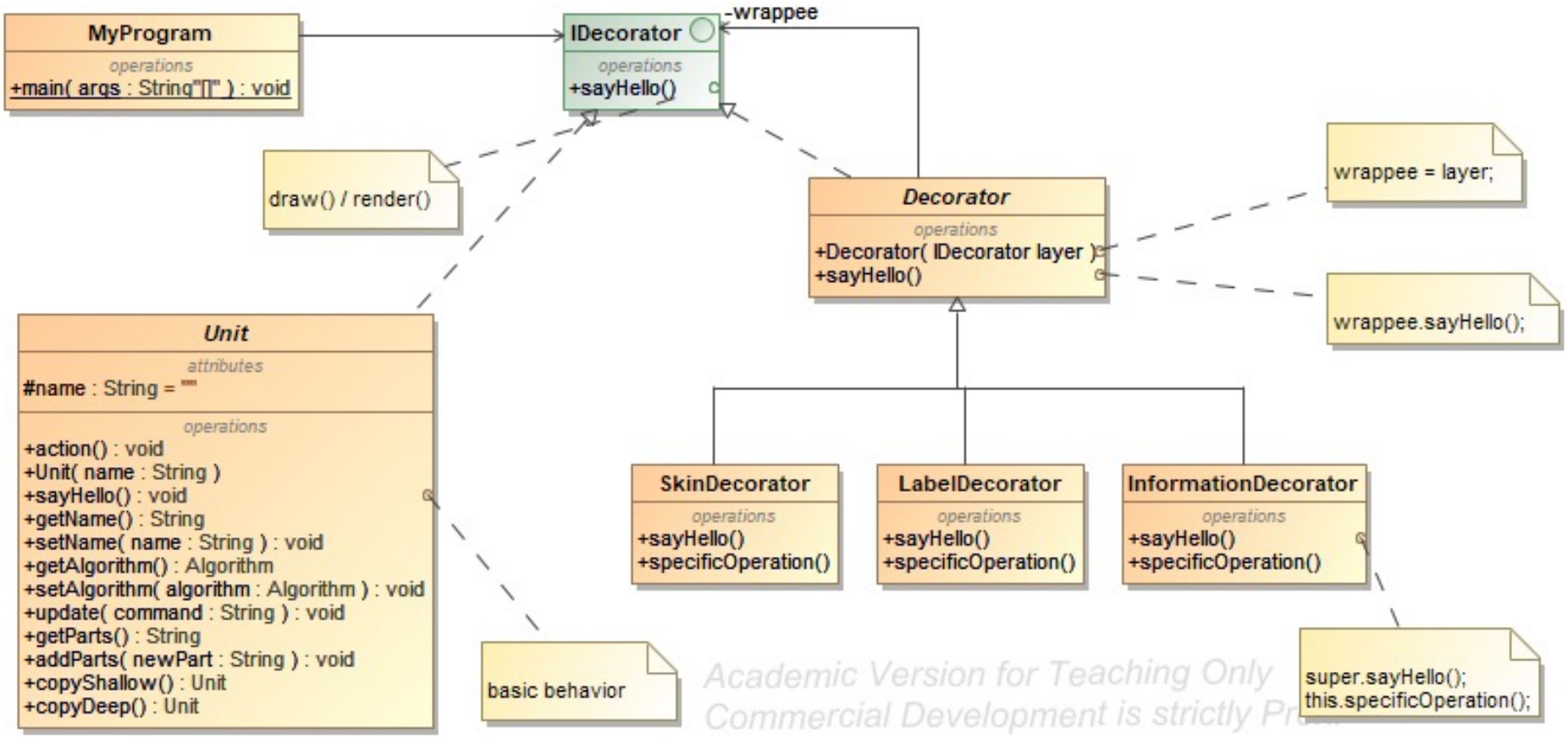
Decorator : example to code

Add Logging, add addition info to the Name

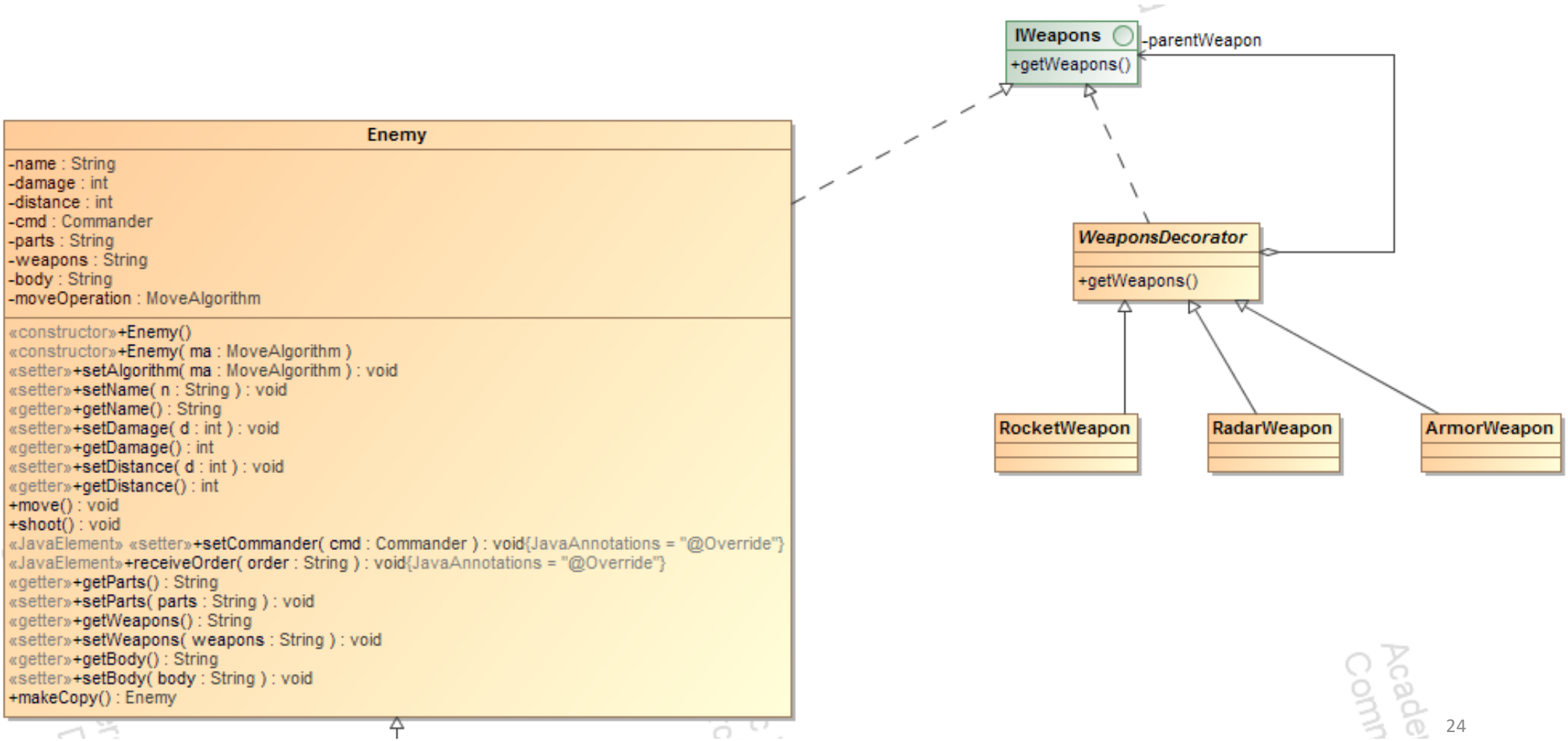
Add to the Unit class decorators:

- *Skin (armor, mask)*
- *Label (name)*
- *Information (Health bar, Energy Bar)*
- *etc.*

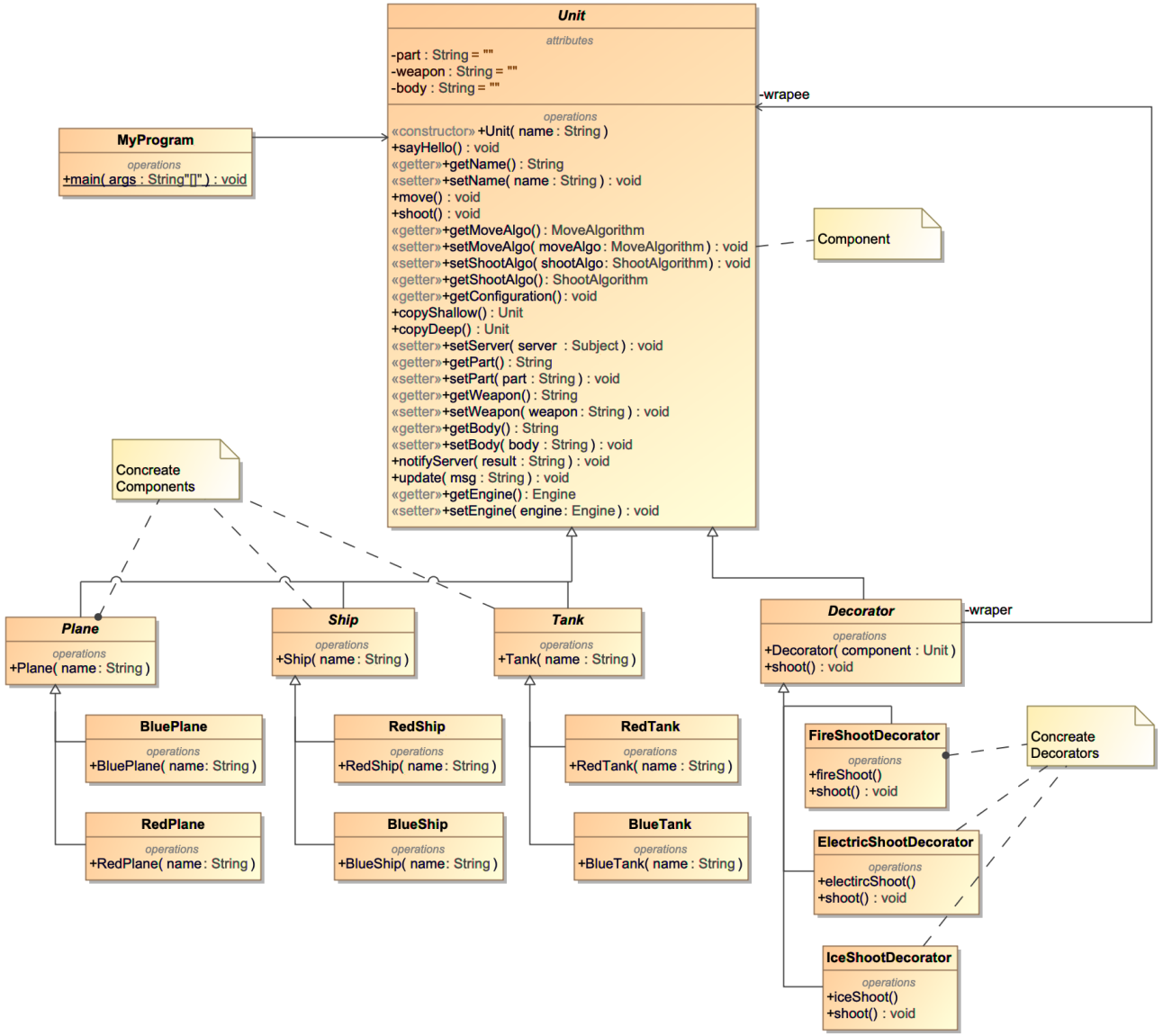
Decorator Example



Decorator example

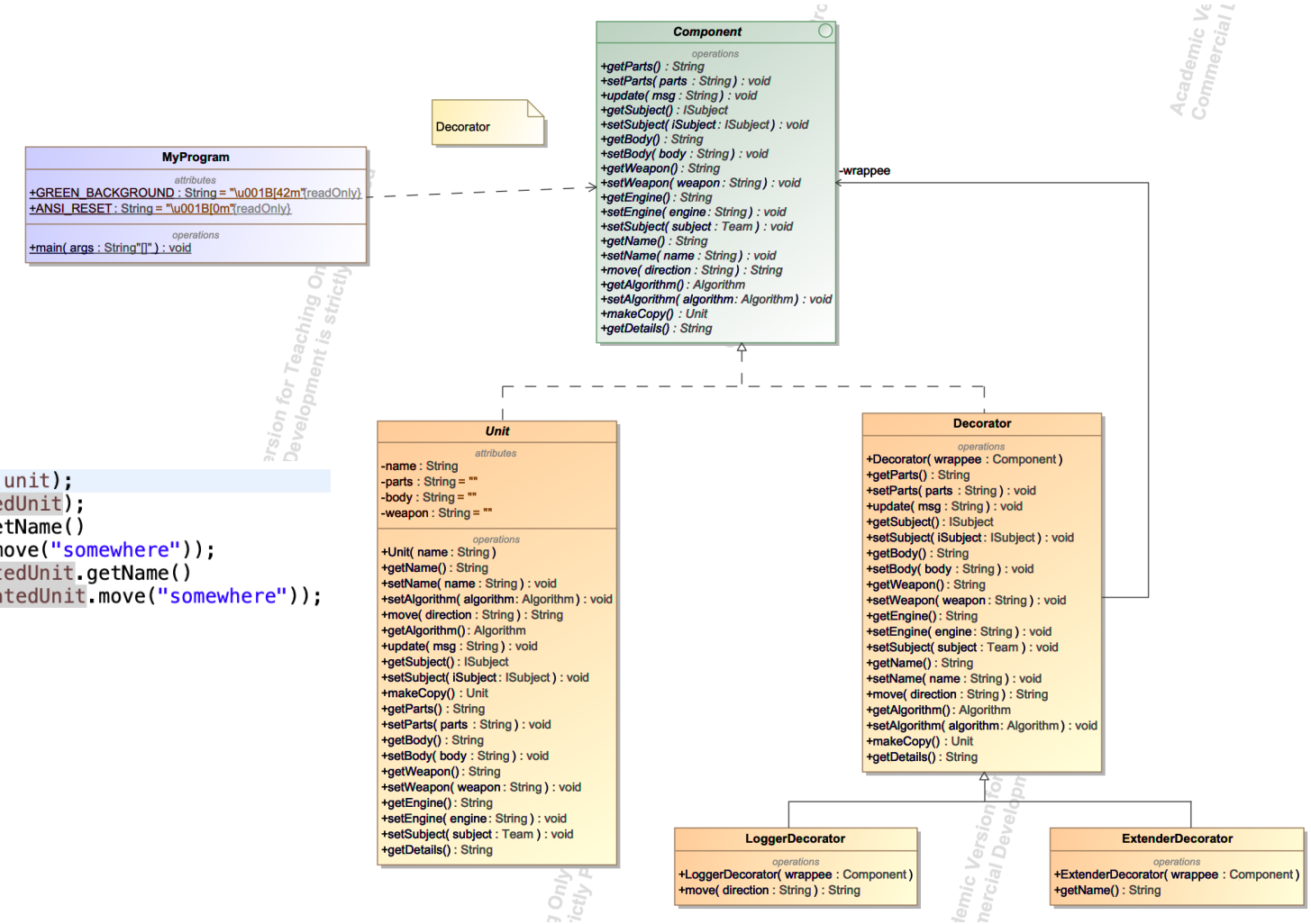


Decorator example



Decorator implementation

```
Component decoratedUnit = new LoggerDecorator(unit);
decoratedUnit = new ExtenderDecorator(decoratedUnit);
System.out.println("    " + "Unit:   " + unit.getName()
                   + ", move: " + unit.move("somewhere"));
System.out.println("    " + "Unit:   " + decoratedUnit.getName()
                   + ", move: " + decoratedUnit.move("somewhere"));
```



`plane.move()` veikia dabar, o jei norime išsaugoti, perduoti, vykdyti vėliau ar atšaukti?

```
public class MyProgram{
    public static void main(args String){
    ...
    plane.move();
    ...
    }
}
```

```
public class MoveCommand(){
    ...
    public execute(){
        plane.move();}
}

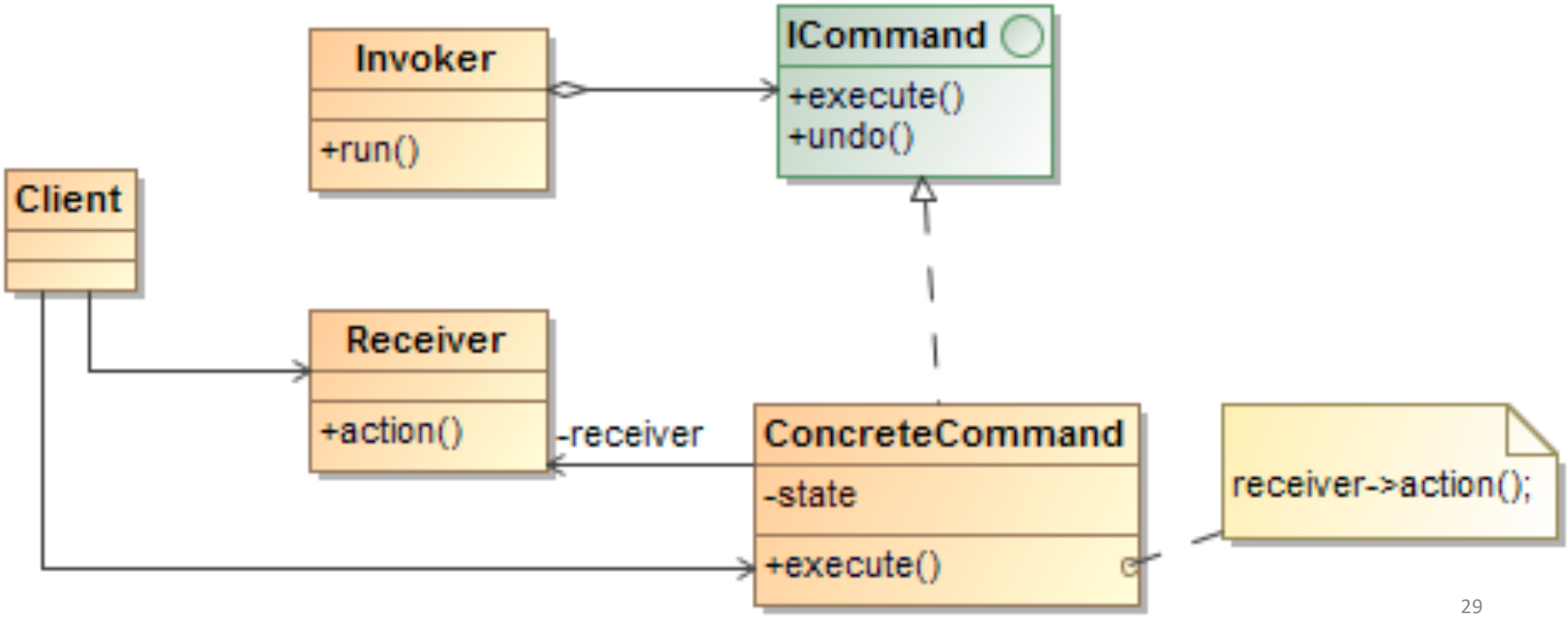
public class MyProgram{
    public static void main(args String)
    {
    ...
    new MoveCommand(plane).execute;
    ...
    }
}
```

Command - Veiksmą paverčiame objektu

- Type: Behavioral
- Also Known As: Action, Transaction
- Intent: “Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support **undo**able operations”

Kaip `Plane.move()` problemą sprendžia GoF

Command's structure



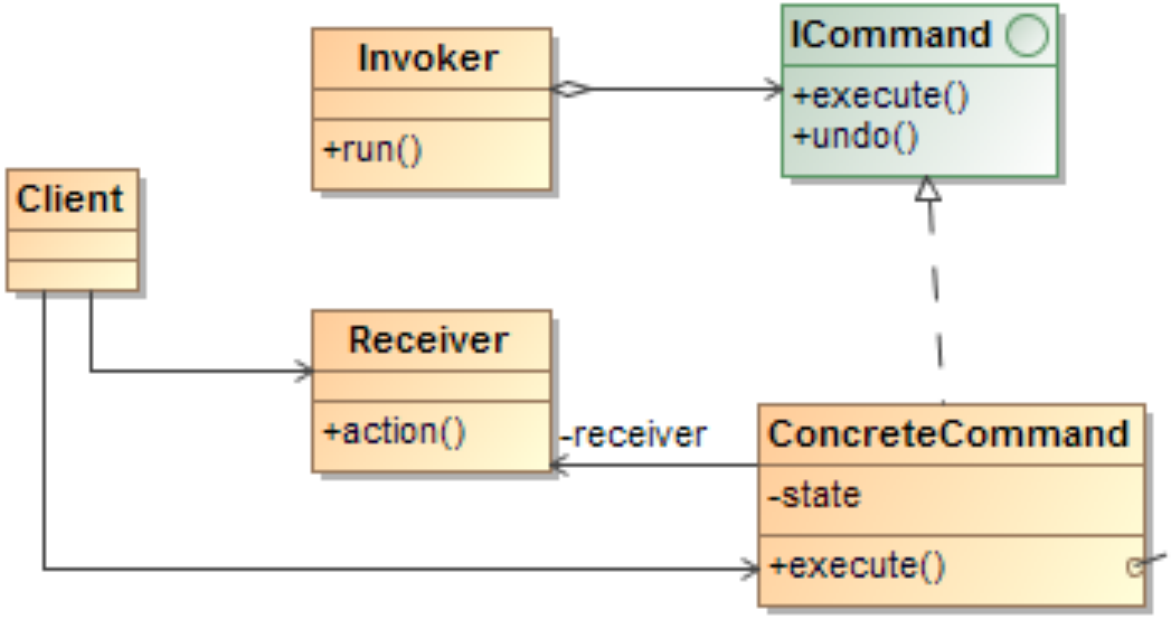
Kas turi prisiminti, kokios Command jau įvykdytos?

A) Receiver

B) Command

C) Invoker

D) Client



Jei žaidėjas įveda 5 komandas iš anksto, kaip sugosime atidėtam vykdymui?

```
RIGHT
RIGHT
SHOOT
LEFT
JUMP
```

A) `String`

B) `Enum`

C) `List<Command>`

Kas bus, jei komandas surinksime į sąrašą `List`?

- Veiksmų istorija,
- Galimybė atlikti Undo() ir Redo()
- Komandų eilę su atidėtu vykdymu

`Plane` pajudėjo `move(right)`.
Kaip padaryti `undo()` ir `redo()`?

A) `move(Left)`

B) išsaugoti ankstesnę poziciją

C) išsaugoti viso Unit kopiją

D) priklauso nuo operacijos

Undo kaip **inverseAction()** arba **stateRestore().**

Prieš **`Command.execute()`** išsaugome Plane kopiją. Kokį jau išmoktą šabloną galime panaudoti?

- Prototype can be used to create Snapshot

Ar dvi MoveCommand yra tokios pačios?

```
MoveCommand(ship, x=10, y=20)

MoveCommand(plane, x=50, y=70)
```

Taigi, kada mums praverčia Command?

- reikia atskirti **kas prašo** nuo **kas vykdo**;
- veiksmą norime **atidėti**;
- veiksmus norime laikyti **queue/history**;
- reikia **Undo/Redo**;
- norime kombinuoti kelias komandas.

Kodėl visur nenaudojame Command?

✅ Sender nepriklauso nuo Receiver realizacijos

✅ Undo / Redo

✅ Queue / delayed execution

✅ lengva pridėti naujas komandas

⚠️ labai paprastam veiksmui atsiranda nemažai papildomų klasių

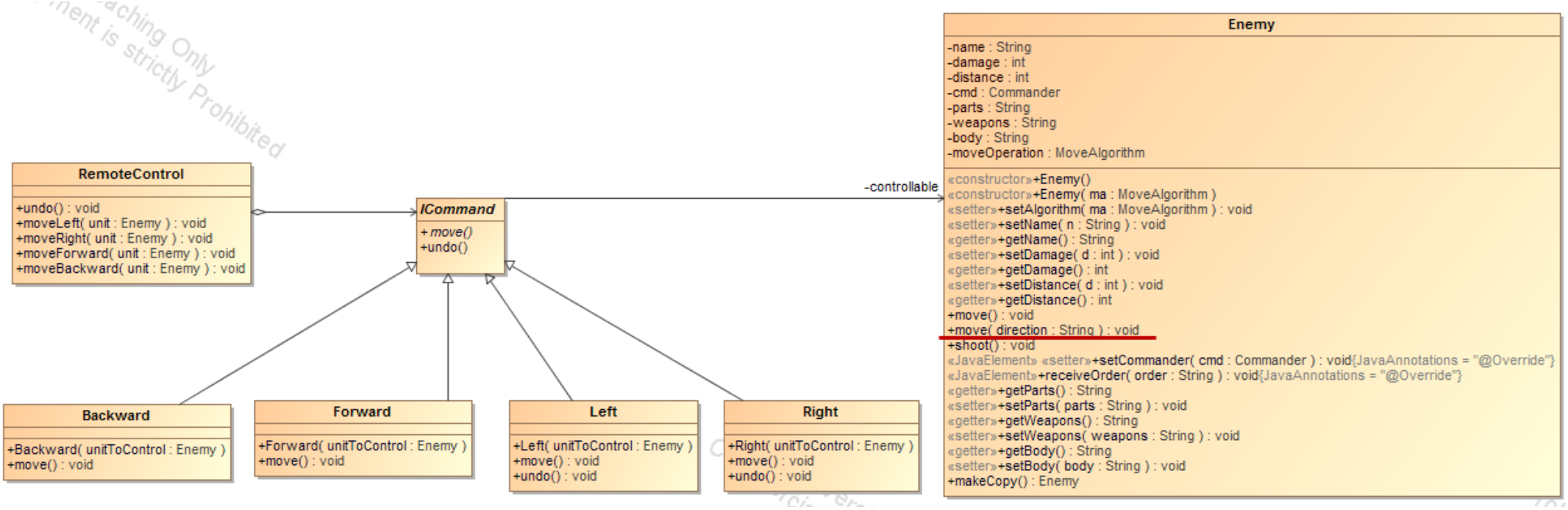
⚠️ Undo gali būti labai sudėtingas

⚠️ reikia valdyti history / command lifetime

Command : example to code

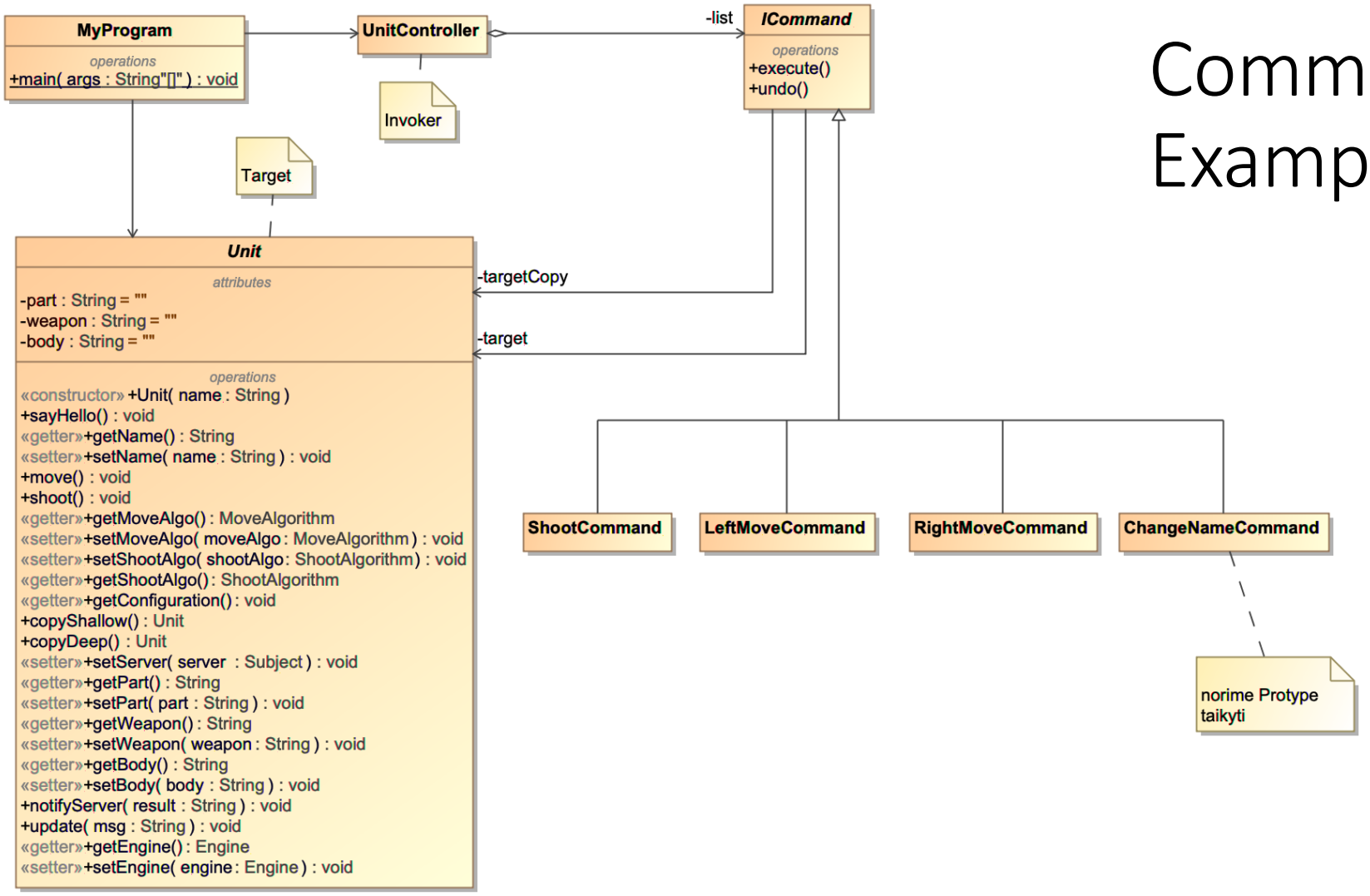
- Apply RemoteControl to different types of units/enemies
  - Declare interface Receiver and apply it to Enemy
  - Add commands: Connect, Left, Right, Forward, Backward, Disconnect.
  - Add Invoker to implement Undo using Java LinkedList<Command>

Command Example

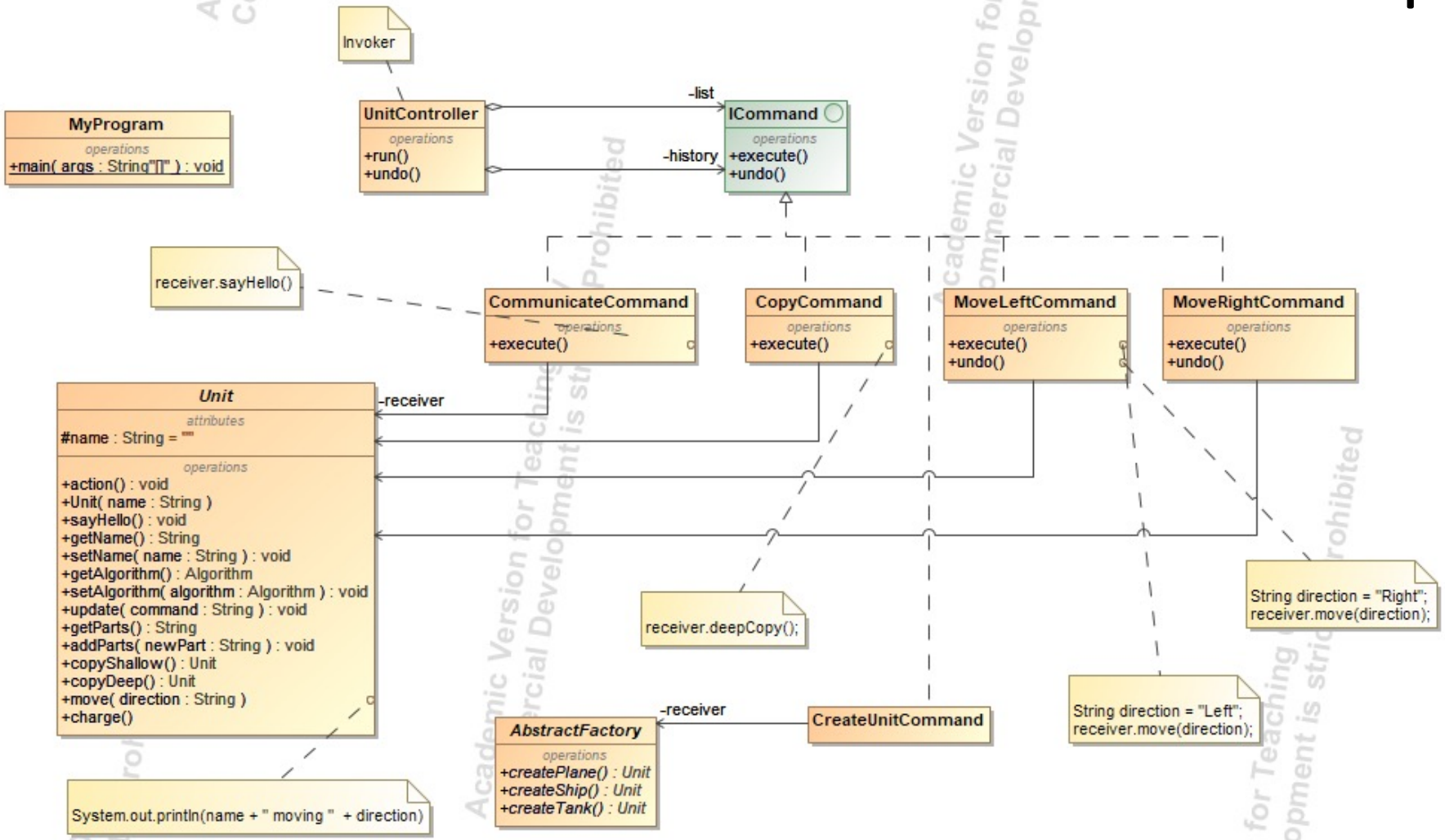


- Added new method `void move(String direction)`
- method `undo()` overrided in Left and Right classes

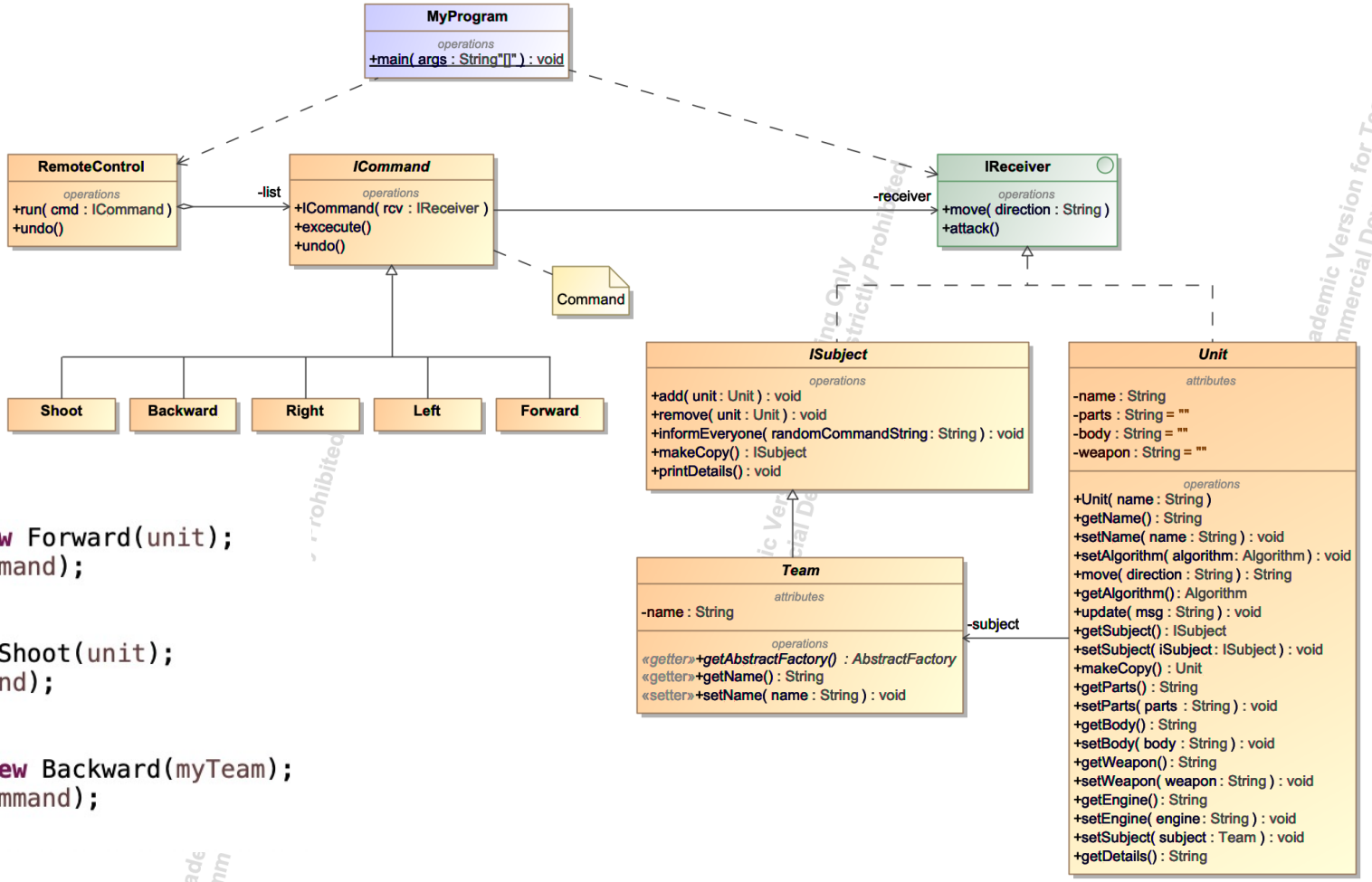
Command Example



Command Example



Command implementation

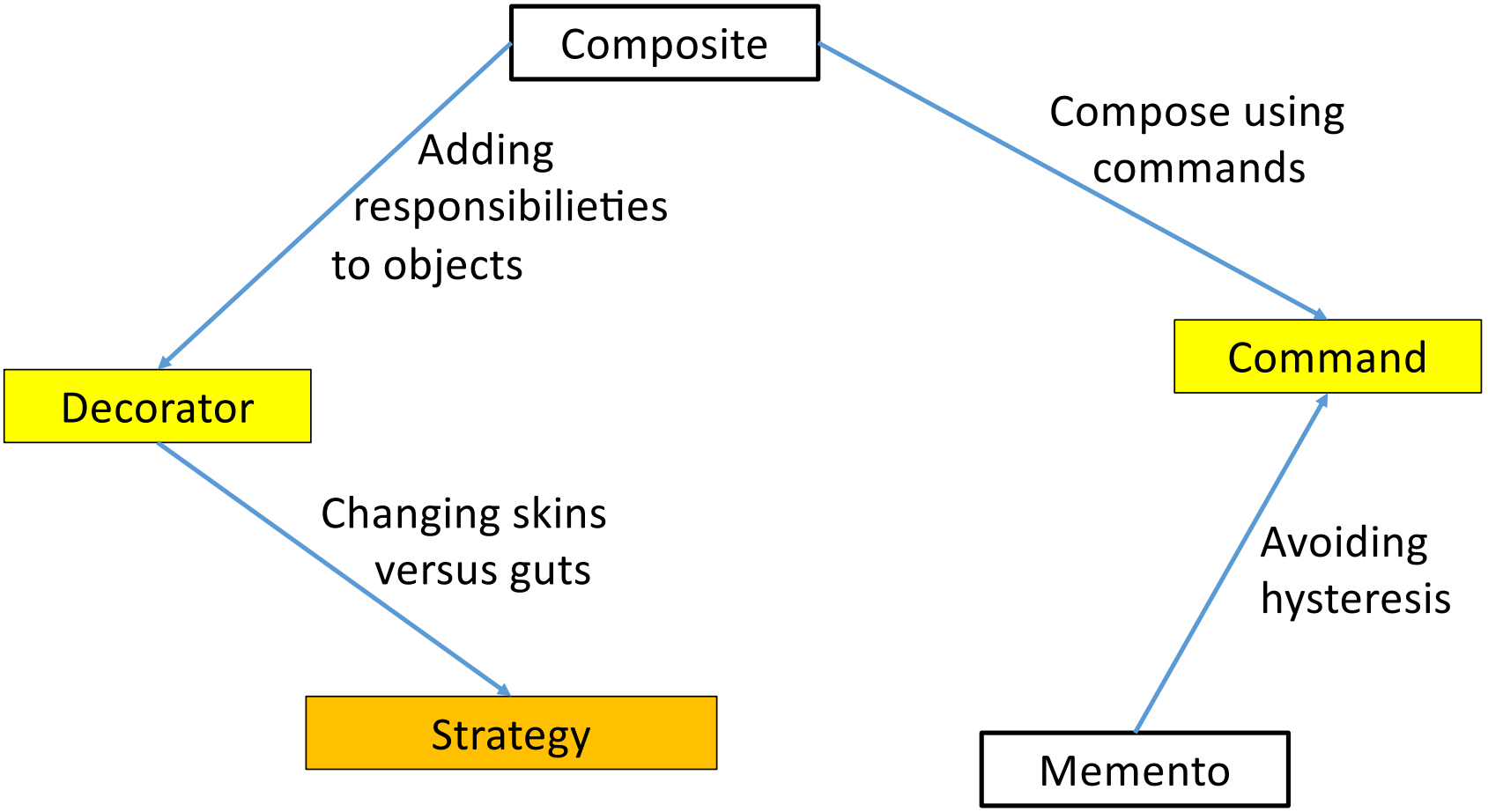



```
ICommand forwardCommand = new Forward(unit);
remoteControl.run(forwardCommand);
remoteControl.undo();

ICommand shootCommand = new Shoot(unit);
remoteControl.run(shootCommand);
remoteControl.undo();

ICommand backwardCommand = new Backward(myTeam);
remoteControl.run(backwardCommand);
remoteControl.undo();
```

Related patterns



Ką šiandien išmokome?

- **Decorator** - keičia tai, KAS vyksta aplink veiksmą
  - Turime veiksmą → pridedame jam naujų sluoksnių.
  - `Logging(Security(Unit))`

- **Command** - paverčia patį VEIKSMĄ objektu
  - Turime veiksmą → paverčiame jį objektu.
  - `MoveRight → Queue → Execute → Undo`

Klausimai?

